

Table of Contents

1. Create a Counter Component
2. Implement a Toggle Switch
3. Build a To-Do List
4. Fetch Data from an API
5. Create a Search Bar
6. Build a Dropdown Menu
7. Implement a Tabs Component
8. Create a Modal Component
9. Build a Carousel Component
10. Implement a Star Rating Component
11. Create a Real-Time Search Filter
12. Build a Multi-Step Form
13. Implement a Virtualized List
14. Create a Reusable Form Component with Validation
15. Implement a Dynamic Form with Field Arrays
16. Implement a Context API for Global State
17. Create a Custom Hook
18. Build a Todo List with Drag-and-Drop
19. Create a Countdown Timer
20. Implement Formik with Yup Validation Conclusion

1. Create a Counter Component.

Problem:

Create a simple counter component that increases or decreases the count when clicking buttons.

Solution:

```
import React, { useState } from 'react';
```

```
const Counter = () => {
```

```
  const [count, setCount] = useState(0);
```

```
  return (
```

```

<div>

  <p>Count: {count}</p>

  <button onClick={() => setCount(count + 1)}>Increase</button>

  <button onClick={() => setCount(count - 1)}>Decrease</button>

</div>

);
};

```

```
export default Counter;
```

2. Implement a Toggle Switch

Problem:

Create a toggle switch component between "On" and "Off" states.

Solution:

```

import React, { useState } from 'react';

const ToggleSwitch = () => {
  const [isOn, setIsOn] = useState(false);

  return (
    <div>

      <button onClick={() => setIsOn(!isOn)}>
        {isOn ? 'Off' : 'On'}
      </button>

    </div>

  );
};

```

```
export default ToggleSwitch;
```

3. Build a To-Do List

Problem:

Create a to-do list component where users can add, remove, and mark items as complete.

Solution:

```
import React, { useState } from 'react';

const TodoList = () => {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState("");

  const addTodo = () => {
    if (text) {
      setTodos([...todos, { text, completed: false }]);
      setText("");
    }
  };

  const toggleTodo = index => {
    const newTodos = [...todos];
    newTodos[index].completed = !newTodos[index].completed;
    setTodos(newTodos);
  };

  const removeTodo = index => {
    const newTodos = [...todos];
    newTodos.splice(index, 1);
    setTodos(newTodos);
  };

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} placeholder="Add a to-do" />
      <button onClick={addTodo}>Add</button>
    </div>
  );
};
```

```

<ul>
  {todos.map((todo, index) => (
    <li key={index} style={{ textDecoration: todo.completed ? 'line-through' : 'none' }}>
      {todo.text}
      <button onClick={() => toggleTodo(index)}>Toggle</button>
      <button onClick={() => removeTodo(index)}>Remove</button>
    </li>
  ))}
</ul>
</div>
);
};

```

export default TodoList;

4. Fetch Data from an API

Problem:

Create a component fetching data from an API and displaying it in a list.

Solution:

```
import React, { useState, useEffect } from 'react';
```

```

const DataFetcher = () => {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);

```

```

  useEffect(() => {
    fetch('https://api.example.com/data')
      .then(response => response.json())
      .then(data => {
        setData(data);
        setLoading(false);
      });
  }, []);

```

```
if (loading) return <p>Loading...</p>;
```

```
return (  
  <ul>  
    {data.map(item => (  
      <li key={item.id}>{item.name}</li>  
    ))}  
  </ul>  
);  
};
```

```
export default DataFetcher;
```

5. Create a Search Bar

Problem:

Create a search bar component that filters a list of items as the user types.

Solution:

```
import React, { useState } from 'react';
```

```
const SearchBar = ({ items }) => {  
  const [query, setQuery] = useState("");  
  
  const filteredItems = items.filter(item =>  
    item.toLowerCase().includes(query.toLowerCase())  
  );  
  
  return (  
    <div>  
      <input  
        type="text"  
        value={query}  
        onChange={e => setQuery(e.target.value)}  

```

```

        placeholder="Search..."
      />
    <ul>
      {filteredItems.map((item, index) => (
        <li key={index}>{item}</li>
      ))}
    </ul>
  </div>
);
};

const App = () => {
  const items = ['Apple', 'Banana', 'Cherry', 'Date', 'Elderberry'];

  return <SearchBar items={items} />;
};

export default App;

```

6. Build a Dropdown Menu

Problem:

Create a dropdown menu component that displays a list of items when clicked.

Solution:

```

import React, { useState } from 'react';

const DropdownMenu = ({ items }) => {
  const [isOpen, setIsOpen] = useState(false);

  return (
    <div>
      <button onClick={() => setIsOpen(!isOpen)}>Menu</button>
      {isOpen && (

```

```

    <ul>
      {items.map((item, index) => (
        <li key={index}>{item}</li>
      ))}
    </ul>
  )}
</div>
);
};

const App = () => {
  const items = ['Profile', 'Settings', 'Logout'];

  return <DropDownMenu items={items} />;
};

export default App;

```

7. Implement a Tabs Component

Problem:

Create a tabs component where each tab displays different content when selected.

Solution:

```

import React, { useState } from 'react';

const Tabs = ({ tabs }) => {
  const [activeTab, setActiveTab] = useState(0);

  return (
    <div>
      <div className="tab-buttons">
        {tabs.map((tab, index) => (
          <button

```

```

      key={index}

      className={index === activeTab ? 'active' : ''}

      onClick={() => setActiveTab(index)}

    >

      {tab.label}

    </button>

  )))
</div>

<div className="tab-content">

  {tabs[activeTab].content}

</div>

</div>

);

};

const App = () => {

  const tabs = [

    { label: 'Tab 1', content: <div>Content of Tab 1</div> },

    { label: 'Tab 2', content: <div>Content of Tab 2</div> },

    { label: 'Tab 3', content: <div>Content of Tab 3</div> },

  ];

  return <Tabs tabs={tabs} />;

};

export default App;

```

8. Create a Modal Component

Problem:

Create a reusable modal component that can be opened and closed and display any content passed to it.

Solution:

```

import React, { useState, useEffect } from 'react';
import ReactDOM from 'react-dom';

const Modal = ({ isOpen, onClose, children }) => {
  useEffect(() => {
    if (isOpen) {
      document.body.style.overflow = 'hidden';
    } else {
      document.body.style.overflow = 'auto';
    }
  }, [isOpen]);

  if (!isOpen) return null;

  return ReactDOM.createPortal(
    <div className="modal-overlay" onClick={onClose}>

    <div className="modal-content" onClick={e => e.stopPropagation()}>
      <button className="modal-close" onClick={onClose}>Close</button>
      {children}
    </div>
  </div>,
    document.body
  );
};

const App = () => {
  const [isModalOpen, setIsModalOpen] = useState(false);

  return (
    <div>

```

```

    <button onClick={() => setIsModalOpen(true)}>Open Modal</button>

    <Modal isOpen={isModalOpen} onClose={() => setIsModalOpen(false)}>

      <h1>Modal Content</h1>

      <p>This is the content inside the modal</p>

    </Modal>

  </div>

);

};

```

```
export default App;
```

9. Build a Carousel Component

Problem:

Create a carousel component that cycles through a set of images.

Solution:

```

import React, { useState } from 'react';

const Carousel = ({ images }) => {

  const [currentIndex, setCurrentIndex] = useState(0);

  const goToNext = () => {

    setCurrentIndex((currentIndex + 1) % images.length);

  };

  const goToPrevious = () => {

    setCurrentIndex((currentIndex - 1 + images.length) % images.length);

  };

  return (

    <div className="carousel">

      <button onClick={goToPrevious}>Previous</button>

      <img src={images[currentIndex]} alt="carousel" />

```

```

    <button onClick={goToNext}>Next</button>
  </div>
);
};

const App = () => {
  const images = [
    'https://via.placeholder.com/600x400?text=Image+1',
    'https://via.placeholder.com/600x400?text=Image+2',
    'https://via.placeholder.com/600x400?text=Image+3',
  ];

  return <Carousel images={images} />;
};

export default App;

```

10. Implement a Star Rating Component

Problem:

Create a star rating component where users can rate something from 1 to 5 stars.

Solution:

```

import React, { useState } from 'react';

const StarRating = ({ totalStars = 5 }) => {
  const [rating, setRating] = useState(0);

  return (
    <div>
      {[...Array(totalStars)].map((star, index) => {
        const starValue = index + 1;
        return (
          <span
            key={index}

```

```

      onClick={() => setRating(starValue)}
      style={{ cursor: 'pointer', color: starValue <= rating ? 'gold' : 'gray' }}
    >
      ★
    </span>
  );
}}
</div>
);
};

```

```

const App = () => {
  return (
    <div>
      <h1>Star Rating</h1>
      <StarRating />
    </div>
  );
};

```

```

export default App;

```

11. Create a Real-Time Search Filter

Problem:

Create a search filter component that filters a list of items in real-time as the user types.

Solution:

```

import React, { useState } from 'react';

const RealTimeSearch = ({ items }) => {
  const [query, setQuery] = useState('');

  const filteredItems = items.filter(item =>

```

```

    item.toLowerCase().includes(query.toLowerCase())
  );

  return (
    <div>
      <input
        type="text"
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Search..."
      />
      <ul>
        {filteredItems.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
};

const App = () => {
  const items = ['Apple', 'Banana', 'Cherry', 'Date', 'Elderberry'];

  return <RealTimeSearch items={items} />;
};

```

```
export default App;
```

12. Build a Multi-Step Form

Problem:

Create a multi-step form where users can navigate between different steps of the form.

Solution:

```
import React, { useState } from 'react';
```

```
const Step1 = ({ next }) => (  
  <div>  
    <h2>Step 1</h2>  
    <button onClick={next}>Next</button>  
  </div>  
);
```

```
const Step2 = ({ next, previous }) => (  
  <div>  
    <h2>Step 2</h2>  
    <button onClick={previous}>Previous</button>  
    <button onClick={next}>Next</button>  
  </div>  
);
```

```
const Step3 = ({ previous }) => (  
  <div>  
    <h2>Step 3</h2>  
    <button onClick={previous}>Previous</button>  
    <button type="submit">Submit</button>  
  </div>  
);
```

```
const MultiStepForm = () => {  
  const [step, setStep] = useState(1);  
  
  const nextStep = () => setStep(step + 1);  
  const previousStep = () => setStep(step - 1);  
  
  const handleSubmit = (e) => {
```

```

    e.preventDefault();
    console.log('Form submitted');
  };

  return (
    <form onSubmit={handleSubmit}>
      {step === 1 && <Step1 next={nextStep} />}
      {step === 2 && <Step2 next={nextStep} previous={previousStep} />}
      {step === 3 && <Step3 previous={previousStep} />}
    </form>
  );
};

const App = () => {
  return (
    <div>
      <h1>Multi-Step Form</h1>
      <MultiStepForm />
    </div>
  );
};

export default App;

```

13. Implement a Virtualized List

Problem:

Create a virtualized list component that efficiently renders a large list of items, only rendering items that are visible within the viewport.

Solution:

```

import React, { useState, useRef, useCallback } from 'react';

const VirtualizedList = ({ items, itemHeight, height }) => {

```

```
const [scrollTop, setScrollTop] = useState(0);  
const totalHeight = items.length * itemHeight;  
const viewportRef = useRef(null);
```

```
const handleScroll = () => {  
  setScrollTop(viewportRef.current.scrollTop);  
};
```

```
const startIndex = Math.floor(scrollTop / itemHeight);  
const endIndex = Math.min(items.length - 1, startIndex + Math.ceil(height / itemHeight));
```

```
const visibleItems = items.slice(startIndex, endIndex + 1).map((item, index) => (  
  <div key={index} style={{ height: itemHeight }}>  
    {item}  
  </div>  
));
```

```
return (  
  <div ref={viewportRef} onScroll={handleScroll} style={{ height, overflowY: 'auto', position: 'relative' }}>  
    <div style={{ height: totalHeight, position: 'relative' }}>  
      <div style={{ position: 'absolute', top: startIndex * itemHeight, width: '100%' }}>  
        {visibleItems}  
      </div>  
    </div>  
  </div>  
);  
};
```

```
const App = () => {  
  const items = Array.from({ length: 1000 }, (_, i) => `Item ${i + 1}`);
```

```

return (
  <div>
    <VirtualizedList items={items} itemHeight={50} height={400} />
  </div>
);
};

```

```
export default App;
```

14. Create a Reusable Form Component with Validation

Problem:

Build a reusable form component that handles form state and validation for various form fields.

Solution:

```
import React, { useState } from 'react';
```

```

const useForm = (initialValues, validate) => {
  const [values, setValues] = useState(initialValues);
  const [errors, setErrors] = useState({});

```

```

  const handleChange = (e) => {
    const { name, value } = e.target;
    setValues({ ...values, [name]: value });

```

```

    const error = validate({ [name]: value });
    setErrors({ ...errors, [name]: error[name] });
  };

```

```

  const handleSubmit = (callback) => (e) => {
    e.preventDefault();
    const validationErrors = validate(values);
    setErrors(validationErrors);

```

```

    if (Object.keys(validationErrors).length === 0) {
      callback();
    }
  };

  return { values, errors, handleChange, handleSubmit };
};

```

```

const Form = ({ onSubmit }) => {
  const initialValues = { username: '', email: '' };

  const validate = (values) => {
    const errors = {};

    if (!values.username) {
      errors.username = 'Username is required';
    }

    if (!values.email) {
      errors.email = 'Email is required';
    } else if (!/^\S+@\S+\.\S+/.test(values.email)) {
      errors.email = 'Email is invalid';
    }

    return errors;
  };
};

```

```

const { values, errors, handleChange, handleSubmit } = useForm(initialValues, validate);

```

```

return (
  <form onSubmit={handleSubmit(() => onSubmit(values))}>
    <div>
      <label>Username</label>
      <input name="username" value={values.username} onChange={handleChange} />

```

```

      {errors.username && <p>{errors.username}</p>}
    </div>
    <div>
      <label>Email</label>
      <input name="email" value={values.email} onChange={handleChange} />
      {errors.email && <p>{errors.email}</p>}
    </div>
    <button type="submit">Submit</button>
  </form>
);
};

```

```

const App = () => {
  const handleSubmit = (values) => {
    console.log('Form Submitted:', values);
  };

  return (
    <div>
      <h1>Reusable Form</h1>
      <Form onSubmit={handleSubmit} />
    </div>
  );
};

```

```
export default App;
```

15. Implement a Dynamic Form with Field Arrays

Problem:

Create a dynamic form that allows users to add or remove fields dynamically.

Solution:

```
import React, { useState } from 'react';
```

```
const DynamicForm = () => {  
  const [fields, setFields] = useState([{} value: ""]);
```

```
  
  const handleChange = (index, event) => {  
    const newFields = fields.slice();  
    newFields[index].value = event.target.value;  
    setFields(newFields);  
  };
```

```
  
  const handleAdd = () => {  
    setFields([...fields, { value: " " }]);  
  };
```

```
  
  const handleRemove = (index) => {  
    const newFields = fields.slice();  
    newFields.splice(index, 1);  
    setFields(newFields);  
  };
```

```
  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    console.log('Form submitted:', fields);  
  };
```

```
  
  return (  
    <form onSubmit={handleSubmit}>  
      {fields.map((field, index) => (  
        <div key={index}>  
          <input  
            type="text"  
            value={field.value}
```

```

      onChange={(e) => handleChange(index, e)}
    />
    <button type="button" onClick={() => handleRemove(index)}>Remove</button>
  </div>
  )}
  <button type="button" onClick={handleAdd}>Add Field</button>
  <button type="submit">Submit</button>
</form>
);
};

```

```

const App = () => {
  return (
    <div>
      <h1>Dynamic Form</h1>
      <DynamicForm />
    </div>
  );
};

```

```
export default App;
```

16. Implement a Context API for Global State

Problem:

Create a global state using React's Context API to manage the state of posts across the application.

Solution:

```
import React, { createContext, useContext, useReducer } from 'react';
```

```
// Create a Context for the posts
```

```
const PostsContext = createContext();
```

```
// Define a reducer to manage the state of posts
```

```
const postsReducer = (state, action) => {  
  switch (action.type) {  
    case 'ADD_POST':  
      return [...state, action.payload];  
    case 'REMOVE_POST':  
      return state.filter((post, index) => index !== action.payload);  
    default:  
      return state;  
  }  
};
```

// Create a provider component

```
const PostsProvider = ({ children }) => {  
  const [posts, dispatch] = useReducer(postsReducer, []);  
  
  return (  
    <PostsContext.Provider value={{ posts, dispatch }}>  
      {children}  
    </PostsContext.Provider>  
  );  
};
```

// Create a custom hook to use the PostsContext

```
const usePosts = () => {  
  return useContext(PostsContext);  
};
```

```
const App = () => {  
  const { posts, dispatch } = usePosts();
```

```
  const addPost = () => {
```

```

    dispatch({ type: 'ADD_POST', payload: 'New Post' });
  };

  const removePost = (index) => {
    dispatch({ type: 'REMOVE_POST', payload: index });
  };

  return (
    <div>
      <button onClick={addPost}>Add Post</button>
      <ul>
        {posts.map((post, index) => (
          <li key={index}>
            {post} <button onClick={() => removePost(index)}>Remove</button>
          </li>
        ))}
      </ul>
    </div>
  );
};

const Root = () => (
  <PostsProvider>
    <App />
  </PostsProvider>
);

```

export default Root;

17. Create a Custom Hook

Problem:

Create a custom hook that fetches and caches data from an API.

Solution:

```
import { useState, useEffect } from 'react';

const useFetch = (url) => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let isMounted = true;

    fetch(url)
      .then((response) => response.json())
      .then((data) => {
        if (isMounted) {
          setData(data);
          setLoading(false);
        }
      })
      .catch((error) => {
        if (isMounted) {
          setError(error);
          setLoading(false);
        }
      });

    return () => {
      isMounted = false;
    };
  }, [url]);

  return { data, loading, error };
};
```

```

const App = () => {
  const { data, loading, error } = useFetch('https://api.example.com/data');

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return (
    <ul>
      {data.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
};

export default App;

```

18. Build a Todo List with Drag-and-Drop

Problem:

Create a todo list application with drag-and-drop functionality to reorder items.

Solution:

```

import React, { useState } from 'react';
import { DragDropContext, Droppable, Draggable } from 'react-beautiful-dnd';

const TodoList = () => {
  const [todos, setTodos] = useState([
    'Learn React',
    'Learn Redux',
    'Build a React App',
  ]);

```

```

const handleOnDragEnd = (result) => {
  if (!result.destination) return;

  const reorderedTodos = Array.from(todos);
  const [removed] = reorderedTodos.splice(result.source.index, 1);
  reorderedTodos.splice(result.destination.index, 0, removed);

  setTodos(reorderedTodos);
};

return (
  <DragDropContext onDragEnd={handleOnDragEnd}>
    <Droppable droppableId="todos">
      {(provided) => (
        <ul {...provided.droppableProps} ref={provided.innerRef}>
          {todos.map((todo, index) => (
            <Draggable key={todo} draggableId={todo} index={index}>
              {(provided) => (
                <li ref={provided.innerRef} {...provided.draggableProps} {...provided.dragHandleProps}>
                  {todo}
                </li>
              )}
            </Draggable>
          )}}
          {provided.placeholder}
        </ul>
      )}
    </Droppable>
  </DragDropContext>
);
};

```

```

const App = () => {
  return (
    <div>
      <h1>Todo List with Drag-and-Drop</h1>
      <TodoList />
    </div>
  );
};

```

```
export default App;
```

19. Create a Countdown Timer

Problem:

Create a countdown timer component that counts down from a given time.

Solution:

```

import React, { useState, useEffect } from 'react';

const CountdownTimer = ({ initialSeconds }) => {
  const [seconds, setSeconds] = useState(initialSeconds);

  useEffect(() => {
    const timer = setInterval(() => {
      setSeconds((prevSeconds) => prevSeconds - 1);
    }, 1000);

    return () => clearInterval(timer);
  }, []);

  return (
    <div>
      <h1>Countdown Timer</h1>
      <p>{seconds} seconds remaining</p>
    </div>
  );
};

```

```

    </div>

  );
};

const App = () => {
  return (
    <div>
      <CountdownTimer initialSeconds={60} />
    </div>
  );
};

export default App;

```

20. Implement Formik with Yup Validation

Problem:

Create a form with validation using Formik and Yup.

Solution:

```

import React from 'react';

import { Formik, Form, Field, ErrorMessage } from 'formik';
import * as Yup from 'yup';

const validationSchema = Yup.object().shape({
  username: Yup.string().required('Username is required'),
  email: Yup.string().email('Invalid email').required('Email is required'),
});

const App = () => {
  return (
    <div>
      <h1>Formik Form with Yup Validation</h1>
      <Formik

```

```

initialValues={{ username: "", email: "" }}
validationSchema={validationSchema}
onSubmit={(values) => {
  console.log('Form Submitted', values);
}}
>
{() => (
  <Form>
    <div>
      <label>Username</label>
      <Field name="username" />
      <ErrorMessage name="username" component="div" />
    </div>
    <div>
      <label>Email</label>
      <Field name="email" type="email" />
      <ErrorMessage name="email" component="div" />
    </div>
    <button type="submit">Submit</button>
  </Form>
)}
</Formik>
</div>
);
};

export default App;

```